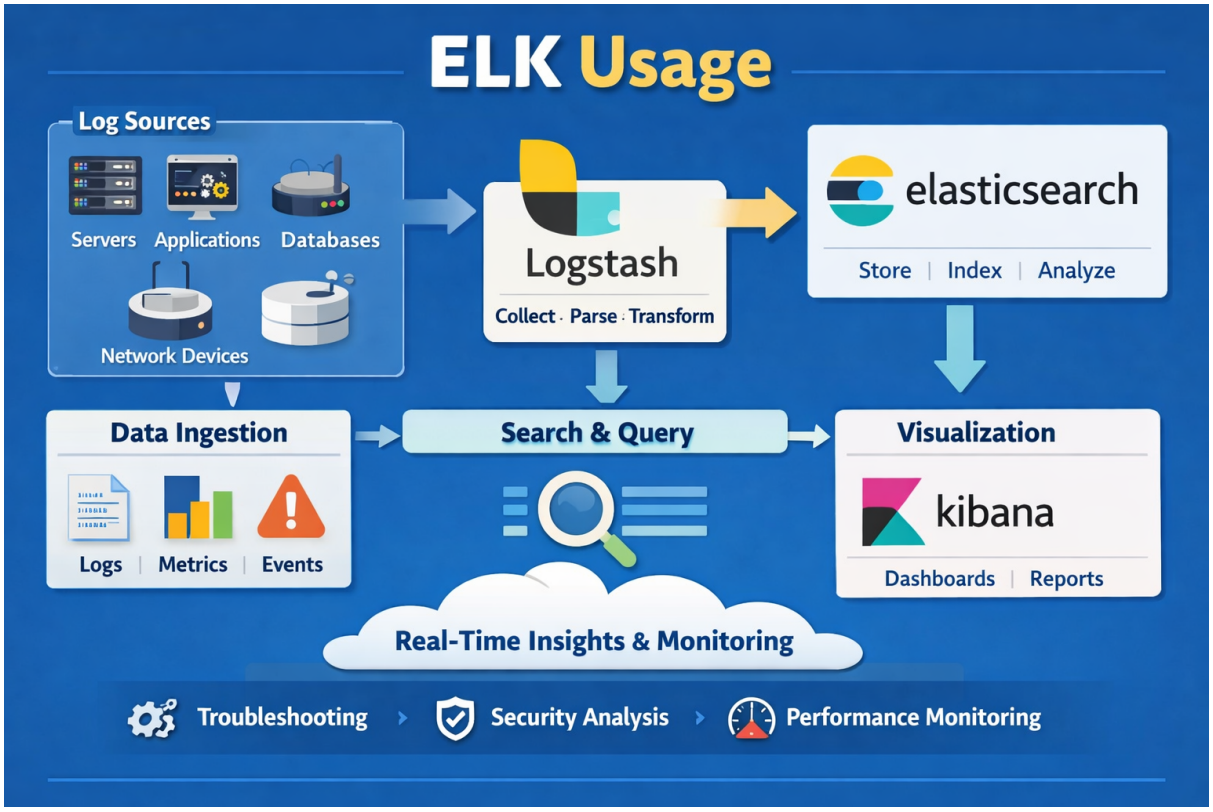
ELK Stack (Elasticsearch, Logstash, Kibana)

Overview of the ELK Stack

The **ELK Stack** is a set of powerful open-source tools designed for searching, analyzing, and visualizing large volumes of data in real-time. The three core components of the stack are:

1. **Elasticsearch** – A distributed search and analytics engine.
2. **Logstash** – A data collection and transformation pipeline.
3. **Kibana** – A visualization and data exploration tool.

The stack is typically used for managing log data but can be extended to handle other types of data such as metrics, business data, and more.



Note: I've included log parsing below to help clarify the concept, as it's crucial to understand before using the term.

Parsing logs - "Parsing logs" refers to the process of analyzing and extracting meaningful information from log files. Log files are records generated by software applications, systems, or servers that track events, processes, and activities, usually for debugging, monitoring, or auditing purposes.

When we "parse" these logs, we're essentially breaking down the raw, often unstructured data into a more structured format, so we can understand, search, and analyze it more easily. Parsing involves:

1. **Identifying log structure**: Logs might follow a specific format (like JSON, XML, or plain text) and contain timestamps, log levels (info, warn, error), message contents, and other metadata.
2. **Extracting data**: You might extract specific fields (like timestamps, error codes, or user IDs) or look for patterns (like specific keywords or repeated errors).
3. **Analyzing logs**: After parsing, you can search or analyze the logs for specific issues, performance bottlenecks, or security breaches.

For example, if a server log contains entries like this:

```
pgsql
2025-05-01 12:30:45 [INFO] User 123 logged in successfully.
2025-05-01 12:31:02 [ERROR] Failed to connect to database.
2025-05-01 12:32:10 [INFO] User 124 logged in successfully.
```

You might parse it to:

- Extract the timestamp
- Separate the log level (INFO, ERROR)
- Identify and act on errors or important events.

The goal is usually to make it easier to spot patterns or diagnose issues, such as monitoring error rates or detecting abnormal activities.

PATHNEX
Shaping Careers, Building Futures

1. Core Components of the ELK Stack

1.1 Elasticsearch

- **What it is**: A distributed, RESTful search and analytics engine.
- **Role**: Stores and indexes data for fast search and real-time analytics.
- **Key Features**:
  - Full-text search using Apache Lucene.
  - Schema-free (data is automatically indexed).
  - Distributed and horizontally scalable.
  - Real-time indexing and searching.
- **How it works**:
  - Data is stored as **documents** in **indices**.
  - Each document is a JSON object and can be queried using the Elasticsearch query language (DSL).
  - It uses **shards** (parts of the index) and **replicas** (copies for redundancy) for scalability and fault tolerance.

1.2 Logstash

- **What it is**: A data processing pipeline that collects, filters, and sends data to Elasticsearch.

- **Role**: Collects, transforms, and forwards data to Elasticsearch for indexing.
- **Key Features**:
  - **Inputs**: Collect data from various sources like logs, files, databases, and external systems.
  - **Filters**: Processes the data to transform it (e.g., parsing logs, extracting fields).
  - **Outputs**: Sends the data to Elasticsearch (or other destinations like a file or a database).
- **Pipeline**:
  - **Input**: Data source (e.g., log files, HTTP, syslog).
  - **Filter**: Transform, parse, or enrich the data.
  - **Output**: Send the data to Elasticsearch for indexing.

1.3 Kibana

- **What it is**: A web-based interface for visualizing and exploring data stored in Elasticsearch.
- **Role**: Provides interactive dashboards and data visualizations for analysis.
- **Key Features**:
  - **Dashboards**: Create interactive dashboards to visualize log data.
  - **Search and Query**: Allows users to search and explore data.
  - **Real-time Analytics**: View and analyze data in real-time.
  - **Visualization**: Create charts, maps, tables, and graphs to present data in meaningful ways.

PATHNEX
Shaping Careers, Building Futures

---

2. How the ELK Stack Works Together

1. **Data Ingestion (Logstash)**:
   - Logstash collects raw data from various sources (e.g., application logs, system logs, metrics).
   - It then applies filters to transform the data into a structured format (e.g., JSON) and prepares it for indexing.
2. **Data Processing (Logstash)**:
   - Logstash can apply powerful filters (e.g., **Grok**, **Date**, **GeoIP**) to parse and enrich the data. For example, it can extract fields like IP addresses, timestamps, and HTTP status codes from unstructured logs.
3. **Data Storage & Indexing (Elasticsearch)**:
   - The transformed data is sent to Elasticsearch, where it is indexed and stored.
   - Elasticsearch stores the data in **indices**, which are made up of **documents**.
   - Documents are JSON objects and are indexed to make them searchable.
4. **Data Visualization (Kibana)**:
   - Kibana provides a user interface to query, explore, and visualize the data stored in Elasticsearch.
   - Users can create dashboards to monitor logs, track application performance, or perform data analysis.

---

3. Step-by-Step Process of Using the ELK Stack

1. **Ingest Data with Logstash**:
   - Logstash collects data from sources like files, databases, or message queues.
   - Example Logstash configuration to read log files:

```
Plaintext

input {
  file {
    path => "/path/to/logfile"
  }
}
```

2. **Process Data in Logstash**:
   - Logstash uses **filters** to parse and structure the raw data. For example, using **Grok filters** to extract structured fields from unstructured log messages:

```
plaintext

filter {
  grok {
    match => { "message" => "%{COMBINEDAPACHELOG}" }
  }
}
```

3. **Store Processed Data in Elasticsearch**:
   - Once Logstash processes the data, it forwards the transformed data to Elasticsearch:

```
plaintext

output {
  elasticsearch {
    hosts => ["http://localhost:9200"]
    index => "web_logs"
  }
}
```

4. **Visualize Data in Kibana**:
   - Kibana connects to Elasticsearch and allows users to create dashboards that visualize the data in real-time.
   - Users can create visualizations like bar charts, pie charts, and line graphs to display trends, outliers, and analytics.

---

4. ELK Stack Architecture

- **Data Sources**: Various sources (e.g., web servers, apps, devices) that generate logs or metrics.
- **Logstash Pipeline**: Collects, filters, and forwards data to Elasticsearch.

PATHNEX
Shaping Careers, Building Futures

    - **Inputs**: Data collection (e.g., logs, HTTP, syslog).
    - **Filters**: Data processing (e.g., Grok, Date).
    - **Outputs**: Data forwarding (to Elasticsearch).
- **Elasticsearch**: Stores and indexes the data.
    - **Indices**: Contain documents (log records).
    - **Shards and Replicas**: Ensure scalability and fault tolerance.
- **Kibana**: Provides a user interface to explore and visualize data from Elasticsearch.

---

5. ELK Stack Use Cases

1. **Log Management**:
    - Collect logs from various systems into a centralized location.
    - Easily monitor system health and troubleshoot errors using Kibana dashboards.
2. **Real-Time Analytics**:
    - Perform real-time analysis of metrics and logs to track performance, detect anomalies, or monitor application behavior.
3. **Security Monitoring**:
    - Analyze security logs for threats, intrusions, or suspicious activities. Create alerts based on patterns or thresholds.
4. **Application Performance Monitoring (APM)**:
    - Monitor the performance of applications, including server uptime, response time, and error rates.
5. **Business Intelligence**:
    - Use Kibana to visualize business metrics, such as sales performance, customer interactions, and other key performance indicators (KPIs).

PATHWAY
Shaping Careers, Building Futures

---

6. How to Set Up the ELK Stack

Step 1: Install Elasticsearch

- Elasticsearch can be installed on Linux, macOS, or Windows using package managers or direct downloads.
- **Example on Linux**:

```
bash

sudo apt-get install elasticsearch
sudo systemctl start elasticsearch
```

Step 2: Install Logstash

- Install Logstash and configure the input, filter, and output settings.
- **Example Input Configuration** (reading logs):

```
plaintext
```

```
input {
  file {
    path => "/var/log/app/*.log"
  }
}
```

Step 3: Install Kibana

- Kibana needs to be installed on a machine that can access your Elasticsearch instance.
- **Example**:

```
bash

sudo apt-get install kibana
sudo systemctl start kibana
```

Step 4: Access Kibana

- Once Kibana is running, open a browser and go to `http://localhost:5601` to access the Kibana dashboard.

PATHNEX
Sharing Careers, Building Futures

7. Best Practices for ELK Stack

1. **Index Management**:
   - Use **index templates** to define the structure of your indices (mappings, settings).
   - Implement **Index Lifecycle Management (ILM)** to manage the retention of old data.
2. **Scalability**:
   - Elasticsearch scales horizontally by adding more nodes to the cluster. Monitor cluster health to avoid performance bottlenecks.
   - Optimize **sharding** and **replication** for large datasets to ensure quick access and high availability.
3. **Data Retention**:
   - Retain only the necessary logs or data in Elasticsearch and set policies for automatic deletion of older logs.
4. **Security**:
   - Use **X-Pack** or **Elastic Security** to enable features like user authentication, encryption, and role-based access control (RBAC).
5. **Monitoring**:
   - Monitor the health of your Elasticsearch and Logstash instances. Use the monitoring features in Kibana for insights into performance.

---

8. Challenges and Considerations

- **Data Volume**: High volumes of data can strain Elasticsearch. Proper scaling, shard management, and index rotation are crucial.
- **Query Complexity**: Complex queries may slow down performance. Use filters and aggregations to optimize queries.
- **Cluster Management**: Running a large Elasticsearch cluster requires good knowledge of node management, backup strategies, and disaster recovery.

📦 In Elasticsearch

1️⃣ Basic Definitions

- **Primary Shards (P)** → Original data partitions
- **Replica Shards (R)** → Copies of primary shards
- **Nodes (N)** → Servers in the cluster

Each primary shard can have **R replicas**.

🔢 Core Formula

✅ Total Shards in Cluster

$$\text{Total Shards} = P \times (R + 1)$$

Where:

- **P** = number of primary shards
- **R** = number of replicas per primary

📌 Example 1

- P = 5
- R = 1

$$5 \times (1 + 1) = 10 \text{ total shards}$$

(5 primary + 5 replica)

PATHNEX
Shaping Careers, Building Futures

📌 Example 2

- P = 3
- R = 2

$$3 \times (2 + 1) = 9 \text{ total shards}$$

(3 primary + 6 replica)

🖥️ Node Requirement Formula

To allocate all shards properly:

Minimum Nodes Required
$= R + 1$

Because:

- A replica cannot be placed on the same node as its primary.
- Each primary + its replicas must be on different nodes.

PATHNEX
Shaping Careers, Building Futures

📌 Example

If:

- R = 2

You need at least:

$$2 + 1 = 3 \text{ nodes}$$

⚖️ Availability Formula

Maximum node failures tolerated:

$$\text{Max Failures} = R$$

Example:

- R = 1 → tolerate 1 node failure
- R = 2 → tolerate 2 node failures

📊 Shard Planning Guideline (Practical Sizing)

Recommended shard size:

- 10–50 GB per shard (general best practice)

Estimated primary shards:

$$P = \frac{\text{Total Data Size}}{\text{Target Shard Size}}$$

Example

Total data = 500 GB
Target shard size = 25 GB

$$P = 500/25 = 20 \text{ primary shards}$$

If R = 1:

$$20 \times (1 + 1) = 40 \text{ total shards}$$

PATHNEX
Shaping Careers, Building Futures

🚨 Important Design Considerations

- Too many shards → memory overhead
- Too few shards → poor parallelism
- Replicas improve:
  - High availability
  - Read performance
- Replicas do NOT improve write performance

🎯 Quick Summary

Concept	Formula
Total shards	P × (R + 1)
Minimum nodes	R + 1
Failure tolerance	R
Primary shard estimate	Data Size ÷ Target Shard Size

📦 Index Lifecycle Management (ILM) in Elasticsearch

Index Lifecycle Management (ILM) automates how indices are managed over time — helping control **cost, performance, and storage** as data ages.

It's mainly used for:

- Logs
- Metrics
- Time-series data
- Security events

PATHNEX
Shaping Careers, Building Futures

🔄 Why ILM Is Needed

Without ILM:

- Indices grow indefinitely
- Storage becomes expensive
- Performance degrades
- Manual cleanup is required

ILM automates:

- Rollover
- Shrink
- Move to cheaper storage
- Delete old data

🏗️ ILM Phases

An index moves through phases based on age or size.

1️⃣ Hot Phase 🔥

- Actively written
- High-performance storage (SSD)
- Full replicas
- Rollover based on:
    - Size (e.g., 50GB)
    - Age (e.g., 7 days)

Example use: Current logs

2️⃣ Warm Phase 🌤️

- Read-only
- Less frequent access
- Can reduce replica count
- Can shrink shard count

Example use: Last month's logs

3️⃣ Cold Phase ❄️

- Rarely accessed
- Cheaper storage
- Possibly fewer replicas
- Searchable but slower

PATHNEX
Shaping Careers, Building Futures

4️⃣ Frozen Phase 🧊 (Optional)

- Very rarely accessed
- Minimal resources
- On-demand loading

5️⃣ Delete Phase 🗑️

- Index automatically deleted
- Frees storage

📊 Example ILM Policy

Scenario:

- Rollover at 50GB or 7 days
- Move to warm after 30 days
- Delete after 90 days

Policy Logic:

```
Hot → 0–30 days
Warm → 30–90 days
Delete → After 90 days
```

🧮 Common Rollover Formula

Rollover triggers when:

```
max_size OR max_age condition met
```

Example:

- max_size = 50GB
- max_age = 7d

Whichever happens first triggers a new index.

PATHNEX
Shaping Careers, Building Futures

🏷️ ILM + Index Naming Pattern

Used with time-based naming:

```
logs-000001
logs-000002
logs-000003
```

New index created automatically during rollover.

Often combined with:

- Data streams
- Templates

⚙️ Benefits of ILM

Benefit	Impact
Automation	No manual cleanup
Cost optimization	Move old data to cheaper storage
Better performance	Smaller active indices
Scalability	Handles TB–PB scale

🧠 Real-World Example

A production logging cluster:

- 100GB logs per day
- Rollover at 50GB
- 2 new indices per day
- Warm after 14 days
- Delete after 180 days

Fully automated — no admin intervention.

PATHNEX
Shaping Careers, Building Futures

🚨 Important Best Practices

- Keep shard size between 10–50GB
- Avoid too many small shards
- Use fewer replicas in warm/cold phases
- Monitor cluster health

🎯 Quick Summary

ILM = Automated index aging strategy

Flow:

```
Hot → Warm → Cold → (Frozen) → Delete
```

It balances:

- Performance
- Availability
- Cost